

Regression Trees

Contents

1. Overview	2
2. When a straight line is not enough	2
2.1 A linear pattern: hours in the semester	2
2.2 A non-linear pattern: hours the day before the exam	3
3. How a regression tree predicts	4
3.1 Cutting the data by hand	4
3.2 A leaf predicts the mean	4
4. Finding the splitting criterion	5
4.1 The question	5
4.2 Scoring one threshold: the sum of squared errors	6
4.3 Trying every threshold	6
4.4 Recursion, and when to stop	7
5. More than one input	8
6. Hyperparameters of a regression tree	9
6.1 max_depth on a non-linear curve	9
7. A regression tree on the Boston housing data	10
7.1 The data and a first tree	11
7.2 Tuning with GridSearchCV and RandomizedSearchCV	11
7.3 Does tuning beat an untuned tree?	12
7.4 The best score of a grid is a little lucky	13
7.5 Feature importance	14
8. Summary	16
Sources	16
9. Key terms	17

Where this fits: Pipeline map steps 5 (Engineer features), 8 (Model), 10 (Tune). See the [Course map](#).



- **Builds on:** Simple imputation (mean, median, mode, constant) ([Note 37](#)); ML pipelines ([Note 38](#)); Regression metrics ([Note 52](#)); Cross-validation ([Note 91](#)); Decision trees ([Note 98](#)); Hyperparameter tuning ([Note 98](#)).
- **Leads to:** Bagging ([Note 105](#)); Random forest ([Note 108](#)); Permutation importance ([Note 114](#)); Gradient boosting ([Note 120](#)).
- **Compare with:** Simple linear regression ([Note 51](#)); Optuna ([Note 134](#)); Bayesian optimisation ([Note 134](#)).

1. Overview

Key point: A regression tree cuts the input space into boxes, exactly like a classification tree, but each leaf predicts the **mean** of its training outputs, and splits are chosen to minimise the squared error.

Decision trees are mostly used for classification (the [decision tree intuition Note](#)), where the output is a category. The same algorithm also works when the output is a number: a **regression tree**.

This Note covers:

- when a regression tree beats linear regression: non-linear data;
- how a tree predicts a number: the mean of a leaf;
- how it finds each split: the sum of squared errors;
- splits with two or more inputs;
- the hyperparameters, and a full example on the Boston housing data, with feature importance.

The Notebook ([notebook.ipynb](#)) runs every example.

2. When a straight line is not enough

Key point: Linear regression handles a roughly straight trend; when the pattern bends, rises and falls, a regression tree follows it.

2.1 A linear pattern: hours in the semester

Key point: More study hours over the semester, more marks, roughly in a straight line.

For many regression problems, linear regression is the go-to algorithm. It finds the **best-fit line** $y = mx + b$ (the [simple linear regression Note](#)) and predicts by reading the line.

Figure 1 (left) plots 30 students: hours studied over the whole semester against exam marks. Some students study less and score more, others the opposite, but the trend is roughly a straight line. The best-fit line predicts that a student who studied 75 hours gets about 60 marks.

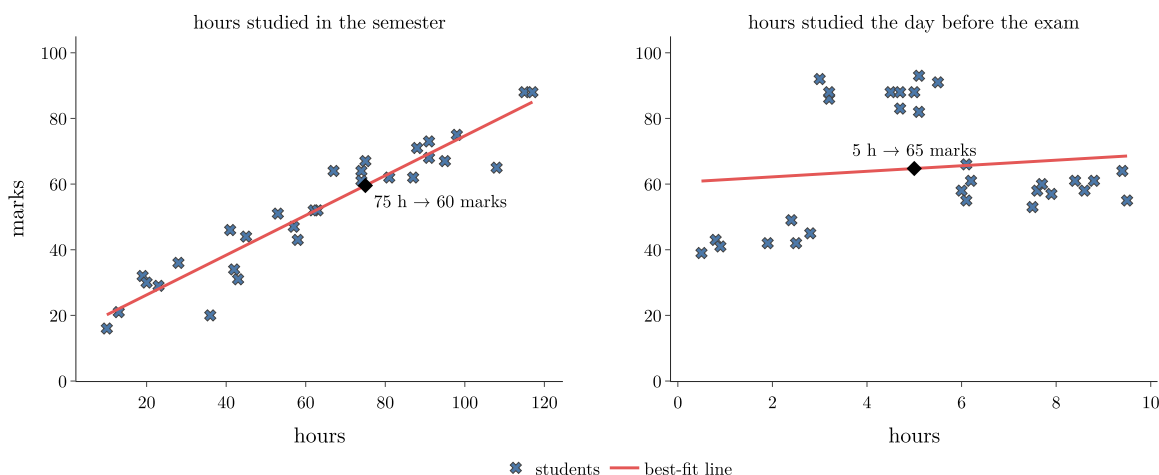


Figure 1: Left: semester hours against marks, roughly linear. Right: hours on the day before the exam against marks, not linear

2.2 A non-linear pattern: hours the day before the exam

Key point: Too little and too much last-day study both score low; a line through this data misses every group.

Figure 1 (right) uses a different input: the hours studied on the day **before** the exam.

- Students who studied **under 3 hours** scored low, around 43.
- Students in the **3 to 6 hours** range scored high, around 88.
- Students who studied **more than 6 hours** scored around 59: they were exhausted on exam day.

A straight line through this data is almost flat. For a student who studied 5 hours, it predicts about 65 marks, while every such student scored above 80. It also misses the low group and the tired group. Its R^2 (the [regression metrics Note](#)) on these 30 students is only 0.02.

Whenever the relationship is non-linear like this, a regression tree usually does better than linear regression.

3. How a regression tree predicts

Key point: Cut the input into ranges with if-else questions; in each range, predict the mean output of the training rows that fall there.

3.1 Cutting the data by hand

Key point: Two cuts, at 3 and 6 hours, separate the three groups.

Decision trees cut the data. Here two cuts are enough: one at 3 hours and one at 6 hours. Written as if-else questions (Figure 2):

1. **Is hours < 6?** If no, the student is in the “tired” group.
2. If yes: **is hours < 3?** If yes, the “too little” group; if no, the “just right” group.

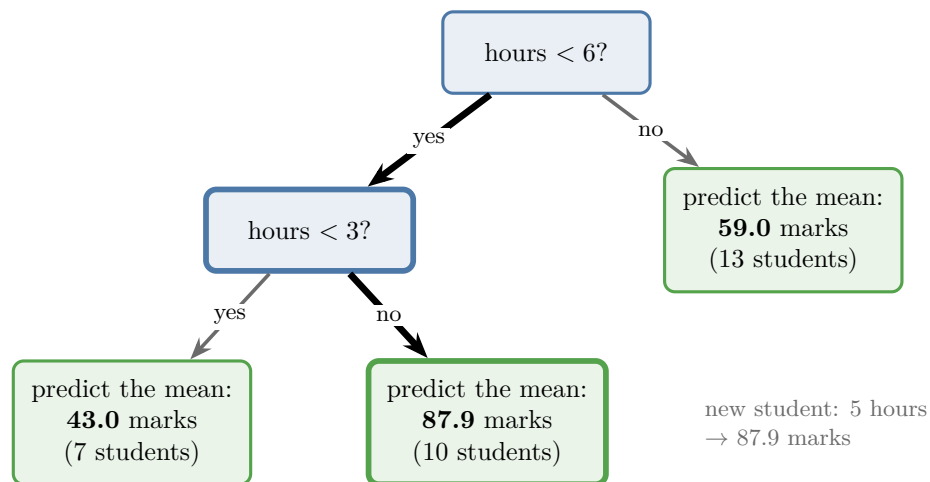


Figure 2: A hand-made regression tree for hours studied the day before the exam

3.2 A leaf predicts the mean

Key point: A leaf’s prediction is the average output of its training rows.

In a classification tree, a leaf predicts its majority class. In a regression tree, a leaf predicts the **mean** output of the training rows that reached it.

1. **In words:** add up the outputs of the rows in the leaf and divide by how many there are.
2. **Formula:** for a leaf with n training rows and outputs $y_1, \dots, y_n$:

$$\hat{y}_{\text{leaf}} = \bar{y} = \frac{1}{n} \sum_{i=1}^n y_i$$

3. **Example:** the 10 students who studied between 3 and 6 hours scored, in total, 879 marks, so the leaf predicts $879/10 = 87.9$.

The three leaves in Figure 2 predict 43.0, 87.9 and 59.0 marks.

A new student who studied 5 hours goes down the tree: $5 < 6$, yes; $5 < 3$, no. They land in the middle leaf, so the tree predicts **87.9 marks**, where the line predicted 65.

Plotted against the hours, the tree's prediction is a staircase: flat within each range, jumping at each cut (Figure 3). scikit-learn, given this data and allowed 3 leaves, finds almost the same cuts by itself: 2.90 and 5.75 hours.

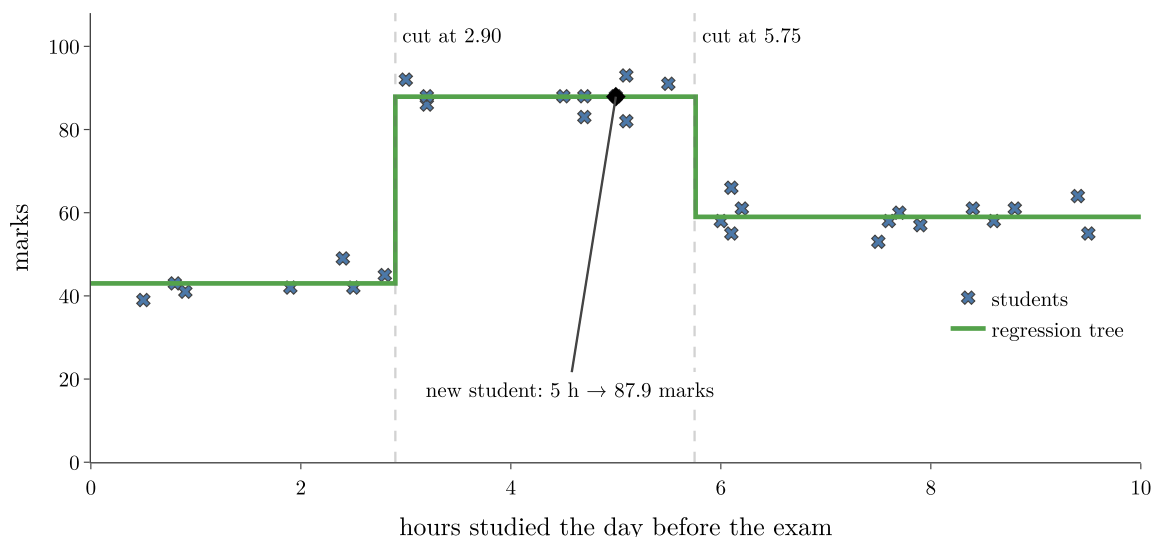


Figure 3: The regression tree's prediction is a step function

4. Finding the splitting criterion

Key point: Try every threshold; for each, predict each side's mean and add up the squared errors; keep the threshold with the smallest total.

4.1 The question

Key point: We can see the gaps at 3 and 6 by eye; the algorithm needs a rule.

In Figure 1 the groups are separated by visible gaps, so we could place the cuts by eye. Real data is continuous, with no gaps, and an algorithm cannot “see”. It needs a rule that scores every possible cut. Classification trees use information gain (the [decision tree intuition Note](#), section 9); regression trees use the squared error.

4.2 Scoring one threshold: the sum of squared errors

Key point: A threshold's score is the sum of squared residuals when each side predicts its own mean.

Take a threshold t between two neighbouring points. It splits the rows into a left group ($x \leq t$) and a right group ($x > t$). Each group predicts its own mean. The gap between a point's actual mark and its group's mean is its **residual** (the [simple linear regression Note](#)), its error.

1. **In words:** square every residual on the left, square every residual on the right, and add them all up. The result is the **sum of squared errors (SSE)** of that threshold.
2. **Formula:**

$$\text{SSE}(t) = \sum_{x_i \leq t} (y_i - \bar{y}_{\text{left}})^2 + \sum_{x_i > t} (y_i - \bar{y}_{\text{right}})^2$$

3. **Example:** 5 students with hours 1, 2, 4, 5, 8 and marks 40, 46, 88, 90, 58. Try $t = 3$ (between 2 and 4):
 - left: 40 and 46, mean 43, so $\text{SSE}_{\text{left}} = (-3)^2 + 3^2 = 18$;
 - right: 88, 90, 58, mean 78.67, so $\text{SSE}_{\text{right}} = 9.33^2 + 11.33^2 + (-20.67)^2 = 642.7$;
 - total: $\text{SSE}(3) = 18 + 642.7 = 660.7$.

The other thresholds score worse: $t = 1.5$ gives 1,443.0, $t = 4.5$ gives 1,880.0 and $t = 6.5$ gives 2,136.0. So $t = 3$ is the best first split for these 5 students.

4.3 Trying every threshold

Key point: Sweep the threshold across the data, plot SSE against it, and take the minimum.

The algorithm repeats this for every candidate threshold:

1. Sort the rows by the input. Take the first two points, and put the threshold halfway between them (their mean). The left group has one point, which is its own mean; the right group has the rest.
2. Compute the SSE and plot it against the threshold.
3. Move to the next two points and repeat, until the threshold has passed every gap.
4. Keep the threshold with the **smallest SSE**. It becomes the split.

Figure 4 shows this sweep on our 30 students ([images/sse_sweep.gif](#) animates it). The red lines are the residuals, the orange and green lines the two means, and the right panel traces the SSE. The minimum is at **hours** ≤ 2.90 , with an SSE of 5,060, down from 9,439 with no split at all.

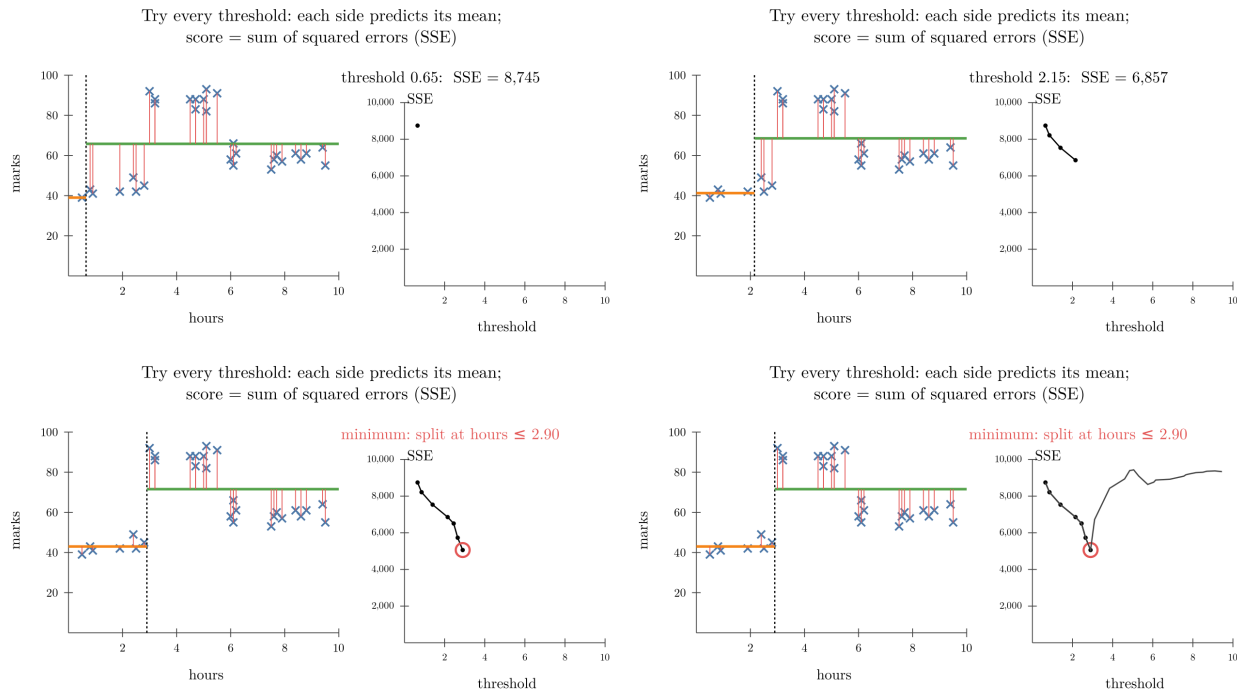


Figure 4: Sweeping the threshold across the data and tracking the SSE (key frames of the animation)

Extra: scikit-learn's default criterion, "**squared_error**", compares splits by the weighted **mean** squared error (MSE) of the children. Since the parent's row count is the same for every candidate, minimising the children's weighted MSE is the same as minimising the SSE. The drop from the parent's MSE to the weighted MSE of its children plays the role that information gain plays in classification; it is often called **variance reduction**, because a node's MSE around its mean is the variance of its outputs (sklearn reference, [criterion](#)).

4.4 Recursion, and when to stop

Key point: Repeat the search inside each side; stop when a node gets too small, or the tree will give every point its own leaf.

After the first split, each side is searched again, in exactly the same way, to find its own best split. On our data the next split, inside the right group, is at 5.75 hours.

If we never stop, the tree keeps splitting until every training point sits alone in a leaf. It then reproduces every training mark exactly but fails badly on new students: overfitting (the [bias-variance Note](#)). We want to separate the groups, not every point.

So we set a stopping rule, for example: do not split a node with fewer than 4 rows. In practice a minimum of about 20 to 25 rows per node is a common starting point, depending on the dataset. This is the hyperparameter `min_samples_split` (or `min_samples_leaf`) of the [hyperparameters Note](#).

5. More than one input

Key point: Find the best threshold for each input separately, then split on the input whose best threshold has the smaller SSE.

Add a second input: each student's CGPA so far. Now there are two inputs, hours and CGPA, and one output, marks, so the data lives in 3D.

For every split, the tree:

1. finds the best threshold on **hours** and its SSE, SSE_{hours} ;
2. finds the best threshold on **CGPA** and its SSE, SSE_{CGPA} ;
3. splits on whichever input has the **smaller** SSE.

In 3D, each split is a plane parallel to one axis that cuts the cloud of points in two. The search then repeats inside each part, with hours and CGPA competing again at every node, until the stopping rule leaves too few rows to split; each final part predicts the mean of its rows.

Figure 5 shows the result on 80 students, viewed from above: each box is a leaf, shaded by its predicted mark. The first split is on hours (≤ 2.8), and the next ones mix hours (at 6.0) and CGPA. Within each range of hours, a higher CGPA means a higher prediction.

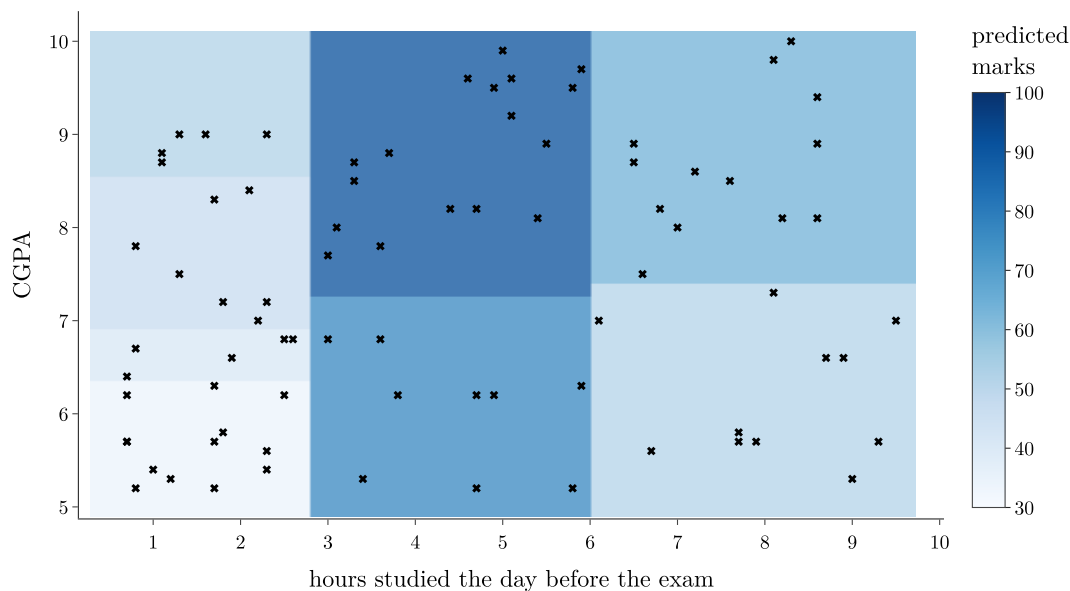


Figure 5: A regression tree with two inputs: each box predicts one mean mark

6. Hyperparameters of a regression tree

Key point: The same knobs as the classification tree; only the criterion changes, from Gini or entropy to squared or absolute error.

`DecisionTreeRegressor` has exactly the hyperparameters of the classifier (the [hyperparameters Note](#)): `splitter`, `max_depth`, `min_samples_split`, `min_samples_leaf`, `max_leaf_nodes`, `min_impurity_decrease` and `max_features`, with the same effects. The difference is the **criterion**, which measures error instead of impurity:

- `"squared_error"` (default): the mean squared error, MSE (the [regression metrics Note](#)); leaves predict the mean.
- `"absolute_error"`: the mean absolute error, MAE; leaves predict the **median**, so outliers pull less (ESL §10.6). The `absolute_error` criterion is slower to train: about 2.4 times slower than `squared_error` on the Boston data in the Notebook.
- `"poisson"`: for counts, such as the number of visits (sklearn UG §1.10.7.2).

On most data `"squared_error"` does as well as or better than the others, but this is again something to settle by tuning.

Extra: Older code also uses `"friedman_mse"`. In scikit-learn 1.9 it is deprecated (to be removed in 1.11) and simply maps to `"squared_error"`, because the two always gave the same trees (sklearn 1.9 deprecation warning).

6.1 `max_depth` on a non-linear curve

Key point: Depth 1 is a single step; depth 5 follows the curve; depth 15 chases every point.

Figure 6 trains trees of four depths on 150 noisy points along a wave and scores them on 50 test points with R^2 .

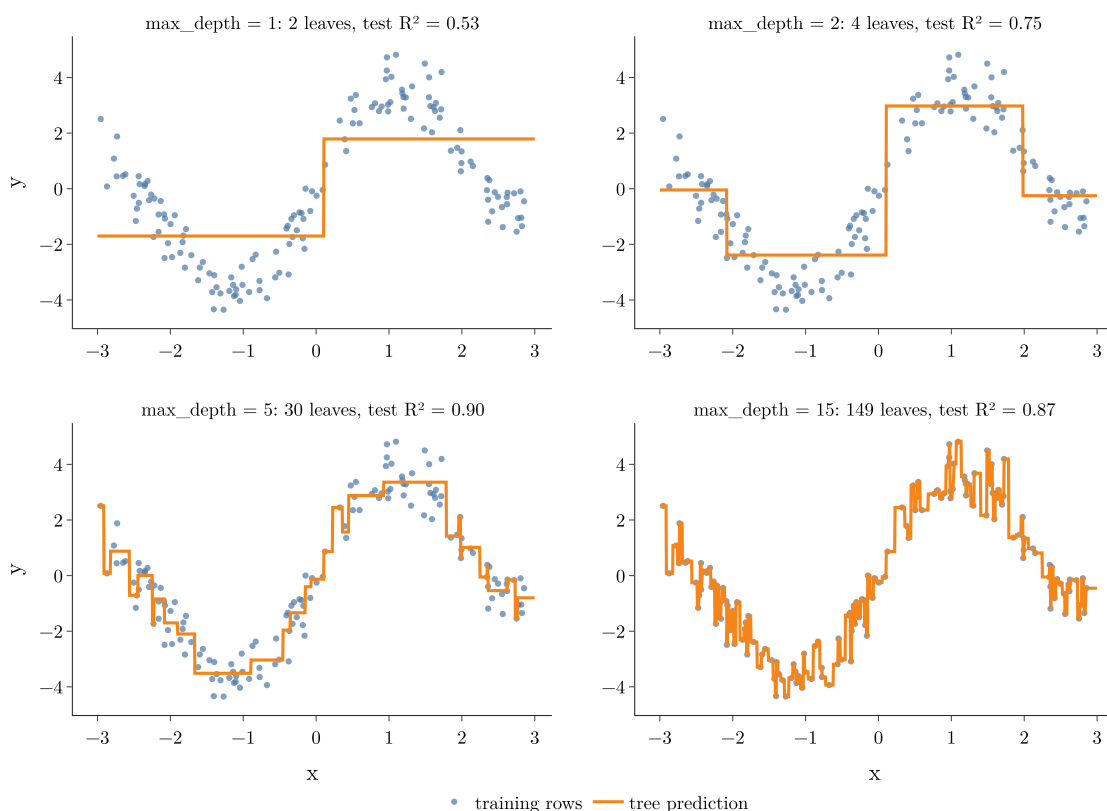


Figure 6: Regression trees of depth 1, 2, 5 and 15 on a noisy wave

- **Depth 1:** one split at $x \approx 0.1$ and two flat levels. Clear underfitting; test $R^2 = 0.53$.
- **Depth 2:** four levels, a rough outline of the wave; $R^2 = 0.75$.
- **Depth 5:** 30 small steps that follow the wave without chasing single points; $R^2 = 0.90$, the best of the four.
- **Depth 15:** 149 leaves for 150 training points. The line jumps to touch almost every point, noise included: overfitting; R^2 falls to 0.87.

The other hyperparameters behave as for classification. For example, with 150 training rows and `min_samples_split=100`, the root splits into 82 and 68 rows, and neither child has 100 rows, so the tree stops there: underfitting. With only one input column, `max_features` has nothing to choose from; it matters on data with many columns.

7. A regression tree on the Boston housing data

Key point: Cross-validation gives an honest score, tuning beats an untuned fully grown tree on new data, and `feature_importances_` shows which features the tree relies on.

7.1 The data and a first tree

Key point: 506 Boston districts, 13 inputs, and the median house price; a depth-5 tree scores $R^2 = 0.88$ on one test split.

The **Boston housing data** describes 506 districts of Boston in the 1970s. Each district is one **observation** (one row of the table). Each has 13 **features**, the input variables (one column each), such as RM (average number of rooms per home), LSTAT (percentage of lower-income residents) and CRIM (crime rate). The **target**, the output we predict, is MEDV, the median home value in thousands of dollars.

Python: A regression tree on the Boston data.

```
import pandas as pd
from sklearn.model_selection import train_test_split
from sklearn.tree import DecisionTreeRegressor
from sklearn.metrics import r2_score

boston = pd.read_csv("data/boston.csv")
X = boston.drop(columns="MEDV")
y = boston["MEDV"]
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42)

reg = DecisionTreeRegressor(criterion="squared_error",
                           max_depth=5, random_state=42)
reg.fit(X_train, y_train)
r2_score(y_test, reg.predict(X_test))    # 0.883
DecisionTreeRegressor is used exactly like DecisionTreeClassifier; only the output
is a number.
```

Extra: Older code loads this data with `load_boston` from `sklearn.datasets`. That function was removed in scikit-learn 1.2, because one column, B, was built from the share of Black residents of each district, an ethically problematic variable (sklearn 1.1, `load_boston` notice). The Notebook reads the same table from `data/boston.csv` (OpenML dataset 531). For new projects, scikit-learn suggests the California housing data instead.

An R^2 of 0.88 looks impressive, but it comes from one random test set of 102 districts. Cross-validation (the [pipelines Note](#), section 8) on the training set gives a more honest average: **0.66**.

7.2 Tuning with GridSearchCV and RandomizedSearchCV

Key point: Grid search tries every combination; randomized search tries a fixed number of random ones, which is faster on big grids.

Grid search with `GridSearchCV` (the [KNN Note](#), section 4.2) tries every combination of the

values we list and keeps the one with the best cross-validated score. Here we tune four hyperparameters:

Python: Tuning a regression tree.

```
from sklearn.model_selection import GridSearchCV

params = {"max_depth": [2, 4, 6, 8, None],
          "criterion": ["squared_error", "absolute_error"],
          "max_features": [0.25, 0.5, 1.0],
          "min_samples_split": [2, 0.05, 0.25]}
grid = GridSearchCV(DecisionTreeRegressor(random_state=42),
                    params, cv=5)
grid.fit(X_train, y_train)
grid.best_params_      # absolute_error, max_depth 6,
                       # max_features 1.0, min_samples_split 2
grid.best_score_       # 0.725, cross-validated R2
A decimal such as max_features=0.5 means “50% of the columns” (6 of 13), and
min_samples_split=0.05 means “5% of the training rows”.
```

That is $5 \times 2 \times 3 \times 3 = 90$ combinations, each cross-validated 5 times: 450 trees. The best combination reaches a cross-validated R^2 of **0.725**.

When a grid becomes too large, **RandomizedSearchCV** is the faster alternative: instead of every combination, it tries `n_iter` combinations drawn at random from the same lists. With `n_iter=20` it trains 100 trees instead of 450 and reaches a cross-validated R^2 of 0.70.

7.3 Does tuning beat an untuned tree?

Key point: An untuned tree grows until every leaf is pure, so it learns the noise. A tuned tree stops earlier and scores clearly better on new data.

With its default settings, **DecisionTreeRegressor** never stops early: it splits until each leaf holds observations with the same target value, often a single observation (section 4.4). Such a tree copies the training data perfectly, noise included. This is why tree size must be tuned: “a very large tree might overfit the data, while a small tree might not capture the important structure”, so the right size should be chosen from the data (ESL §9.2.2).

The effect is clearest on a large dataset with a noisy target. We use the **California housing data**: 20,640 districts of California in 1990, with 8 features (input variables) such as median income and house age, and the median house value as the target (sklearn California housing). Each district is one observation.

The experiment changes one thing only, the tree’s settings:

1. split the data 80/20 into training and test sets;
2. **untuned tree:** fit `DecisionTreeRegressor()` with its defaults on the training set;
3. **tuned tree:** run `GridSearchCV` over `max_depth` (4 to 12, or `None`) and `min_samples_leaf` (1, 5, 20, 50) **on the training set only**, and keep its

- best tree;
4. score both trees on the test set, which the search never saw.

The Notebook repeats this over 20 random splits and averages the scores (Figure 7).

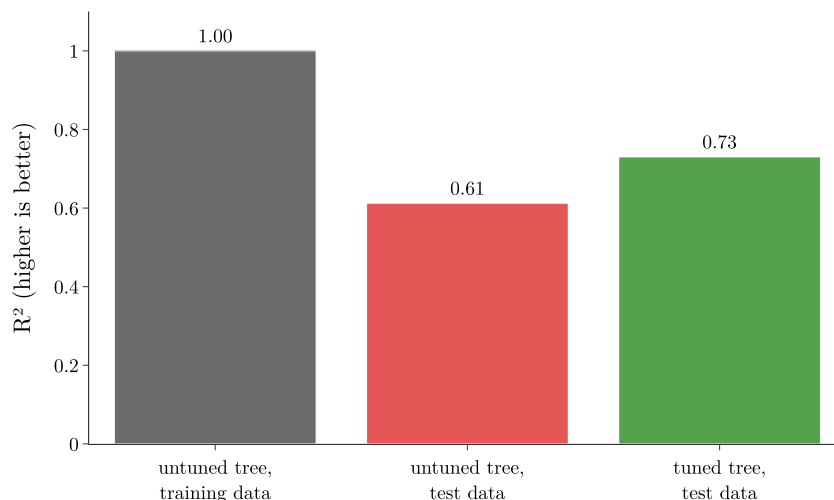


Figure 7: Average R^2 over 20 train-test splits of the California housing data

- The untuned tree scores a perfect $R^2 = 1.00$ on its training data, with about 15,900 leaves for 16,512 training observations: it has memorised the noise.
- On the test data it falls to **0.61**.
- The tuned tree, with about 630 leaves, scores **0.73** on the test data, and it beats the untuned tree on all 20 splits.

So tuning gains about 0.12 in test R^2 here. On every split the search picked `min_samples_leaf=20`: every leaf averages at least 20 districts, which smooths out the noise in single prices.

7.4 The best score of a grid is a little lucky

Key point: The best of many cross-validated scores is slightly too high; re-score the winner on fresh folds to get an honest number.

Picture 90 random people and pick the tallest. That person is tall, but part of the reason they won is luck of who showed up. A grid search works the same way. On the Boston data in section 7.2, it keeps the best of 90 scores, all measured on the same 5 folds, so a setting can win partly because it happens to suit those folds. Picking the best of many scores always makes the winner look a little better than it is; this effect is called **selection bias** (Cawley and Talbot 2010).

The Notebook measures it. On 10 fresh shuffles of 5-fold cross-validation (only the folds change), the Boston grid's winning setting scores **0.663**, not 0.725. To report an honest score,

either re-score the winner on fresh folds, or test it on data the search never saw, as in section 7.3.

7.5 Feature importance

Key point: `feature_importances_` gives each column's share of the tree's total error reduction; on Boston, RM, LSTAT and CRIM dominate.

A trained tree also tells us which columns it relied on. Its attribute `feature_importances_` gives one number per column: that column's share of all the impurity (here, error) reduction achieved by the tree's splits. The numbers add up to 1.

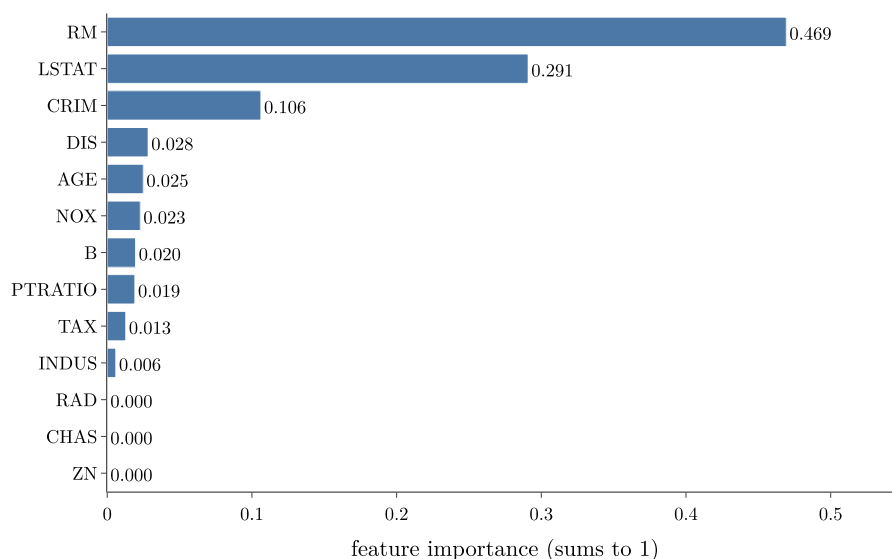


Figure 8: Feature importance of the tuned tree on the Boston data

Figure 8 shows them for the tuned Boston tree of section 7.2:

- **RM** (rooms per home) is by far the most important column, at 0.47;
- then **LSTAT** (0.29) and **CRIM** (0.11);
- the last few columns (**INDUS**, **ZN**, **CHAS**, **RAD**) contribute almost nothing.

This is useful for **feature selection** (the [curse of dimensionality Note](#)): if we must drop columns, the ones with near-zero importance are the first candidates.

Extra: The importance of a column is computed by adding up, over every node that splits on it, the node's share of the training rows times its impurity decrease (the Δ of `min_impurity_decrease` in the [hyperparameters Note](#), section 4.8), and then dividing by the total over all columns. A single tree is unstable: a small change in the data can give very different splits (ESL §9.2.4), so its importances can change a lot from one training set to another. A random forest averages them over many trees and gives steadier values; in the Notebook, over 30 resampled training sets, the spread of RM's

importance drops from 0.20 for one tree to 0.12 for a forest.

8. Summary

	Classification tree	Regression tree
Output	a class	a number
Leaf predicts	the majority class	the mean of its rows (median with <code>absolute_error</code>)
Split chosen by	information gain (Gini or entropy)	smallest squared error (SSE / MSE)
scikit-learn	<code>DecisionTreeClassifier</code>	<code>DecisionTreeRegressor</code>
Criterion values	"gini", "entropy", "log_loss"	"squared_error", "absolute_error", "poisson"
Prediction shape	boxes of one class	a staircase: boxes of one value

- Linear regression handles roughly straight trends; regression trees handle non-linear ones, such as marks against last-day study hours.
- Each candidate threshold splits the data in two; each side predicts its mean; the threshold with the smallest SSE wins (here, hours ≤ 2.90).
- With several inputs, each input's best threshold competes; the smallest SSE wins.
- A tree grown fully gives every point its own leaf; stop it with `max_depth`, `min_samples_split`, `min_samples_leaf` and the other hyperparameters.
- An untuned tree grows until its leaves are pure and memorises the noise; on California housing, tuning `max_depth` and `min_samples_leaf` raises the test R^2 from 0.61 to 0.73 (average of 20 splits).
- The best score of a grid is a little optimistic (best of 90): on fresh folds the Boston winner scores 0.663, not 0.725.
- On the Boston data, RM, LSTAT and CRIM are the most important features.

Sources

- **Cawley and Talbot 2010:** G. C. Cawley and N. L. C. Talbot, "On Over-fitting in Model Selection and Subsequent Selection Bias in Performance Evaluation", *Journal of Machine Learning Research* 11, 2010.
- **ESL:** T. Hastie, R. Tibshirani and J. Friedman, *The Elements of Statistical Learning*, 2nd ed., Springer, 2009. Sections 9.2.2, 9.2.4 and 10.6.
- **sklearn California housing:** scikit-learn User Guide, Real world datasets, California Housing dataset (`fetch_california_housing`); data from R. K. Pace and R. Barry, "Sparse Spatial Autoregressions", *Statistics and Probability Letters* 33, 1997.
- **sklearn UG:** scikit-learn User Guide, Section 1.10.7.2, Regression criteria.
- **sklearn reference:** scikit-learn `DecisionTreeRegressor` API reference, parameter `criterion`.
- **sklearn 1.9 deprecation warning:** the warning raised by `DecisionTreeRegressor(criterion="fr` in scikit-learn 1.9.

- **sklearn 1.1, load_boston notice:** scikit-learn 1.1 API reference for `sklearn.datasets.load_boston` (deprecated in 1.0, removed in 1.2).

9. Key terms

Term	Meaning
Regression tree	A decision tree whose leaves predict numbers: the mean output of their training rows
DecisionTreeRegressor	scikit-learn's regression tree
Sum of squared errors (SSE)	The sum of the squared residuals; a regression tree splits where the SSE of the two sides is smallest
Variance reduction	The drop in mean squared error from a node to its children; the regression version of information gain
squared_error	DecisionTreeRegressor's default criterion: split by mean squared error, leaves predict the mean
absolute_error	A criterion that splits by mean absolute error; leaves predict the median
RandomizedSearchCV	Tuning that cross-validates a fixed number of randomly drawn hyperparameter combinations
Feature importance	A column's share of all the impurity reduction in a tree; the shares add up to 1
feature_importances_	The fitted attribute holding the feature importance of every column
California housing data	20,640 California districts (1990), 8 features and the median house value; built into scikit-learn
Selection bias	The best of many scores looks better than it really is, because part of its win is luck
Boston housing data	506 Boston districts, 13 inputs and the median home value; removed from scikit-learn in version 1.2